Механизм: Limited Direct Execution

Для того чтобы виртуализировать CPU, ОС нуждается каким-то образом делить физический CPU между множеством задач запущенных одновременно. Основная идея проста: запустить один процесс ненадолго выполняться на процессоре, затем запустить другой и так далее. Разделяя CPU по времени таким образом виртуализация и достигается.

Однако, при таком подходе существуют определенные трудности. Первая из них - производительность: как мы можем осуществлять виртуализацию без добавления перерасхода ресурсов системы? Следующая проблема - контроль: как запускать другие процессы эффективно, в то время как процесс удерживает контроль над CPU? Контроль над CPU - это очень важная часть работы ОС, поскольку именно она ответственна за ресурсы. Без контроля процесс может работать бесконечно, завладев всеми ресурсами машины, или получать доступ к информации, к которой при нормальных условиях у него не должно быть доступа. Таким образом Одной из главных сложностей построения операционных систем является получение большой производительности, сохраняя при этом контроль над системой.

Задача: как эффективно виртуализировать CPU, сохраняя контроль.
Операционная система должна виртуализировать CPU эффективно, сохраняя при этом контроль над системой. Чтобы этого добиться, одновременно нужна поддержка и оборудования и операционной системы. Таким образом операционная система будет часто использовать некоторые возможности оборудования для того, чтобы выполнять свою работу эффективно.

базовая техника: limited Direct execution

Неудивителен тот факт, что разработчики операционных систем в погоне за наибольшей производительностью пришли к технике, которая называется limited Direct execution. Та часть, которая относится к прямому выполнению довольно-таки проста: просто начните выполнять программу прямо на центральном процессоре. Таким образом, когда операционная система желает запустить программу выполняются следующие действия:

- создается запись о программе в списке процессов
- выделяется некоторое количество памяти
- код программы загружается с жесткого диска в ОЗУ
- определяется точка входа (например функция main())
- выполняется переход к выбранной точке входа программы
- начинается выполнение кода пользователя.

OS	Program
Create entry for process list	
Allocate memory for program	
Load program into memory	
Set up stack with argc/argv	
Clear registers	
Execute **call** main()	
	Run main()
	Execute **return** from main
Free memory of process	
Remove from process list	

Direct execution Protocol (W/O limits)

На схеме виден упрощенный процесс протокола прямого исполнения (пока без каких либо ограничений). Использование функции вызова (call) и возврата (return) для перехода к функции main() в программе и последующего возвращения из этой функции обратно в ядро.

Звучит довольно просто, однако при таком подходе существуют некоторые проблемы. Например, если мы просто запустим программу, то как ОС убедиться, что эта программа не делает ничего такого, чего бы мы не хотели, при этом сохраняя эффективность работы этой программы. Другая проблема заключается в том, что во время того как процесс запущен, ОС необходимо остановить его и переключиться на выполнение другого процесса, таким образом воплощая разделение по времени, которое требуется для виртуализации CPU.

Отвечая на эти вопросы будет понятно откуда взялась ограничивающая часть в названии (limited). Без ограничений запускаемых программ, ОС не сможет контролировать что-либо и превратится всего лишь в библиотеку.

Проблема №1: Ограниченные Операции

Прямое выполнение имеет очевидное преимущество - оно быстро. Программа запускается как есть на физическом CPU и таким образом выполняется с ожидаемой скоростью - скоростью процессора. Но исполнение на CPU подводит нас к проблеме: а что если процесс захочет выполнить какую-то операцию из списка ограниченных? Например обратиться к устройству ввода-вывода или получение доступа к ресурсам типа CPU или ОЗУ?

Суть проблемы: как выполнять ограниченные операции
Процесс должен иметь возможность выполнять запросы к устройствам ввода-вывода, но должен делать это не получая полный контроль над системой. Как же ОС и оборудование смогут совместно решить эту задачу?

Одно из возможных решений данной проблемы - давать возможность процессу делать все что ему вздумается в рамках I/O и подобных операций. Однако, подобное решение лишает нас множества необходимого функционала. Например, если бы мы хотели разработать файловую систему, которая бы проверяла права на доступ к файлу до того как обратившийся к этому файлу получал бы к нему доступ, ни в коем случае нельзя было бы давать возможность любому пользователю выполнять I/O к диску. Если бы мы разрешили, процесс мог бы просто прочитать или записать на диск. И таким образом никакие ограничения бы не подействовали.

Поэтому один из сложившихся подходов к решению подобных проблем - новое состояние процессора, так называемое пользовательский режим (user mode). Код, который выполняется в пользовательском режиме ограничен в своих возможностях. Так например код, исполняемый в пользовательском режиме уже не может выполнять операций I/O - на подобные обращения процессор будут вызывать исключения, а операционная система - убивать подобный процесс.

В противовес к пользовательскому режиму существует режим ядра (kernel mode), в котором работает сама ОС, а точнее ее ядро. В этом режиме исполняемый код может делать все что ему вздумается, включая привилегированные операции, такие как I/O или выполнение любого типа ограниченных инструкций.

Режим ядра и пользователя однако не решает задачу с предоставлением процессу возможности выполнить привилегированную операцию, такую как I/O. Чтобы разрешить эту операцию программе, все современное оборудование предоставляет возможность пользовательским программам осуществлять так называемые системные вызовы (system call). Впервые представлены на таком допотопном оборудовании как Atlas, системные вызовы позволяют ядру аккуратно предоставлять некоторые кусочки функциональности пользовательским программам, такие как доступ к файловой системе, создание и уничтожение процесса, взаимодействие с другими процессами, выделение дополнительной памяти. Большинство ОС предоставляют сотни системных вызовов. Ранние UNIX системы предоставляли всего порядка 12 вызовов.

Чтобы выполнить системный вызов, программа должна выполнить специальную инструкцию, называемую trap (ловушка). Эта инструкция одновременно переходит в ядро и повышает уровень привилегий в режим ядра. Находясь в режиме ядра, система теперь может выполнять любые привилегированные операции, которые ей нужны и таким образом выполнять требуемую работу для процесса, запросившего эту операцию. После окончания выполнения задачи ОС вызывает специальную инструкцию возвращения (return-from-trap), которая в свою очередь возвращается в пользовательскую программу и одновременно с этим понижает уровень привилегий до пользовательского.

Оборудование должно выполнять операцию перехвата аккуратно, поскольку необходимо сохранить состояние регистров вызывающей программы, чтобы корректно их восстановить при возвращении обратно к программе после вызова функции return-from-trap. Для архитектуры x86, например, процессор запишет (push) Program Counter, флаги и некоторое количество других регистров в стеке ядра - для каждого процесса будут сохранены значения регистров этого процесса. Функция возврата из перехвата (return-from-trap) затем извлечет (pop) и восстановит эти значения и далее

выполнение процесса пользователя продолжится. В других системах могут использоваться другие конвенции, но базовый принцип между различными платформами будет сохраняться.

Остается еще один важный вопрос, как функция перехвата узнает, какой код выполнять в ОС? Вызывающий процесс не может указать конкретный адрес перехода, поскольку иначе это позволит программам выполнять переходы куда угодно и такое решение крайне плохо скажется на безопасности. И поэтому ядру необходимо контролировать код, который выполняется после осуществления перехвата.

Для выполнения операций перехвата в ядре реализован механизм, называемый таблицей перехвата (trap table), которая заполняется в момент загрузки ОС. Когда ОС загружается, она находится в привилегированном режиме (режиме ядра) и таким образом свободно может конфигурировать оборудование машины так как необходимо. Одна из первых вещей которые делает ОС - передает оборудованию информацию о том, какой код запускать, когда наступает некоторое событие. Например, какой код нужно запускать, когда случается прерывание жесткого диска, клавиатуры или программа делает системный вызов. ОС информирует оборудование об местонахождении обработчиков ловушек (trap handler) с помощью специальных инструкций. Оборудование запомнит эту информацию до следующей перезагрузки. Таким образом теперь оборудование знает какой код исполнять, когда происходят системные вызовы или иные исключающие события.

Для того чтобы выполнить конкретный системный вызов, каждому вызову присваивается номер (system call number). И таким образом пользовательский код ответственен за помещение нужного номера системного вызова в регистр или в необходимое место в стеке. В момент обработки системного вызова внутри обработчика ловушки, ОС проверяет этот номер, убеждается в его правильности и если это так, исполняет необходимый код. Такая косвенность обращения формирует некоторую защиту - пользователь не может указать конкретный адрес перехода, но может указать на нужный код через его номер.

--- Отступление --

И хотя мы избавились от множества болей с помощью механизмов системных вызовов, ловушек и таблицы системных вызовов, все еще остается множество вопросов безопасности, например необходимо проверять параметры, передаваемые системному вызову. ОС должна убеждаться что они правильные, в противном случае отклонить такой вызов.

Например при осуществлении вызова write(), пользователь должен указать на адрес буфера в качестве источника. Если пользователь укажет плохой адрес (намеренно или случайно), например внутри адресного пространства ядра, ОС должна это определить и отклонить такой вызов. В противном случае пользователь сможет считывать всю память ядра. Поскольку память ядра в том числе включает в себя всю физическую память системы, эта лазейка позволит также читать память всех остальных процессов в системе.

Поэтому очень важно проверять пользовательский ввод и относиться к нему всегда с большим подозрением. Недостаточное фильтрование пользовательского ввода - одна из популярных техник нахождения уязвимостей практически в любой системе.

OS @ boot (kernel mode)	Hardware	
initialize trap table		
	remember address of... syscall handler	

OS @ run (kernel mode)	Hardware	Program (user mode)
Create entry for process list		
Allocate memory for program		
Load program into memory		
Setup user stack with argv		
Fill kernel stack with reg/PC		
return-from-trap		
	restore regs from kernel stack	
	move to user mode	
	jump to main	
		Run main()
		...
		Call system call
		trap into OS
	save regs to kernel stack	
	move to kernel mode	
	jump to trap handler	
Handle trap		
Do work of syscall		
return-from-trap		
	restore regs from kernel stack	
	move to user mode	
	jump to PC after trap	
		...
		return from main
		trap (via `exit()`)
Free memory of process		
Remove from process list		

Figure 6.2: **Limited Direct Execution Protocol**

Рассмотрим схематично как выполняется протокол LDE. В него входит 2 фазы: в первую очередь (в момент загрузки) ядро инициализирует таблицу перехвата, а CPU запоминает ее местонахождение. Ядро выполняет эту операцию в привилегированном режиме. Каждая привилегированная операция выделена жирным.

Во время следующей фазы (во время исполнения процесса) перед тем как выполнить возврат из перехватчика(trap) и начать выполнение программы, ядро выполняет подготовку (запись о процессе в таблице процессов, выделение памяти). Затем происходит возврат (return-from-trap), режим процессора меняется на пользовательский и начинается исполнение процесса. Когда процесс пытается выполнить системный вызов, он перехватывается (traps) и выполняется ОС, а затем возвращается к выполнению процесса вновь после вызова возврата (return-from-trap). Процесс завершая свою работы возвращается из функции main(). Обычно после этого выполняется некоторый служебный код, который правильно завершит программу, например выполнив системный вызов exit(), который снова будет перехвачен ОС. В это время ОС завершает процесс, очищая его ресурсы и данные.

Проблема 2: Переключение между процессами

Следующая проблема с прямым исполнением - достижение переключения между процессами. Переключение между процессами должно осуществляться просто. ОС должна остановить выполнение одного процесса и запустить другой, всего навсего. На самом деле звучит гораздо проще, чем есть на самом деле. Когда процесс исполняется на CPU, это означает что в данный момент ОС не запущена. Если ОС не запущена, как мы вообще можем что-либо делать? И это становится реальной проблемой: Если ОС не исполняется на процессоре, она не может совершать никаких действий.

Суть проблемы: как возвращать контроль над CPU? Как ОС будет возвращать контроль над CPU, так чтобы она смогла переключать работу процессов?

Кооперативный подход: ожидание системных вызовов

Один из подходов ОС прошлого (Ранние версии Макинтош или Xerox Alto) называется кооперативным подходом. В этой архитектуре ОС верит, что процессы в системе ведут себя разумно. Предполагается, что процессы работающие долго будут периодически освобождать CPU и позволять ОС решать какой процесс запустить далее..

Каким же образом такие дружелюбные процессы в некотором утопическом мире будут освобождать процессор? Большинство процессов будет передавать контроль над процессором, осуществляя системные вызовы, например для открытия файла или создания нового процесса. Подобные системы часто включают в себя системный вызов yield(), который не делает ничего, только лишь передает управление ОС.

Приложения также передают управление ОС, когда они пытаются сделать что-то противоправное, например попытка поделить на 0 или попытка получения доступа к памяти, к которой не должно быть этого доступа, то такое поведение приведет к перехвату управления ОС. ОС вновь получит контроль над CPU (и возможно завершит этот процесс).

Получается, что в кооперативных системах, ОС ожидает получения контроля над CPU и получает его только тогда, когда происходит какой-то легальный системный вызов

или какая-то нелегальная (ошибочная) операция. Эта архитектура ничего не сможет противопоставить программе, которая входит в бесконечный цикл. ОС будет бесконечно ждать CPU, но никогда его не получит.

Не кооперативный подход: ОС получает контроль

Получается, что без помощи оборудования ОС не сможет сделать ничего, если процесс не сделает ни одного системного вызова (или ошибку) и таким образом возвратив контроль ОС. Фактически в кооперативных системах единственное решение против таких процессов - перезагрузка всей ОС.

> Суть проблемы: как вернуть контроль без кооператива.
> Как ОС вернуть контроль над CPU даже если процессы не кооперативны? Как ОС убедиться, что никакой процесс не получит постоянный контроль над машиной?

Ответ на эти вопросы довольно прост и был придуман много лет назад: прерывание по времени. Устройство-таймер может быть запрограммировано вызывать прерывание каждые столько-то миллисекунд. Когда вызывается прерывание, текущий процесс останавливается и запускается заранее сконфигурированный обработчик прерываний. В этот момент ОС получает контроль на ОС и дальше работать согласно планировщику - остановить работу текущего процесса и запустить новый.

При загрузке ОС оборудование также как и с таблицей системных вызовов уведомляется о том как запустить обработчик прерываний. Также при старте ОС будет запущен и сам таймер. Запуск таймера также является привилегированной операцией. После того как таймер будет запущен, ОС может спокойно передавать управление, поскольку будет уверена что получит его обратно.

Поведение предпринимаемое после вызова прерывания таймером похоже на системный вызов - точно также нужно сохранить состояние регистров процессора, так чтобы позже можно было к нему вернуться.

Сохранение и Восстановление Контекста

Теперь когда ОС восстанавливает контроль над системой, не важно кооперативно, через системный вызов или через таймер, нужно принять решение - продолжать ли текущий запущенный процесс или переключиться на другой процесс. Это решение принимается через часть ОС которая называется планировщик.

Если принятое решение - переключиться, ОС выполняет низкоуровневую часть кода, которая называется контекстное переключение (context switch). Концепция контекстного переключения проста - все что нужно сделать ОС это сохранить состояния регистров для текущего запущенного процесса (например в стек ядра) и затем нужно восстановить состояние регистров для запланированного процесса (из стека ядра). Выполняя это действие ОС убеждается что после возвращения из перехвата (return-from-trap) вместо возвращения к процессу, который был запущен ранее, система будет исполнять какой-то другой процесс.

Для сохранения контекста текущего процесса, ОС выполняет некоторое количество низкоуровневого ассемблерного кода, который сохраняет основные значения регистров, PC, Stack Pointer, а затем на эти же позиции восстанавливает значения будущего процесса.
В момент переключения стеков ядро начинает выполнение системного вызова, переключающего код в контексте одного процесса (который был прерван) и возвращается из вызова в контексте уже другого процесса. Когда ОС наконец выполняет функцию возврата (return from trap), то этот самый другой процесс становится текущим исполняемым процессом и на этом контекстное переключение завершено.

OS @ boot (kernel mode)	Hardware	
initialize trap table		
	remember addresses of...	
	syscall handler	
	timer handler	
start interrupt timer		
	start timer	
	interrupt CPU in X ms	

OS @ run (kernel mode)	Hardware	Program (user mode)
		Process A
		...
	timer interrupt	
	save regs(A) to k-stack(A)	
	move to kernel mode	
	jump to trap handler	
Handle the trap		
Call `switch()` routine		
save regs(A) to proc-struct(A)		
restore regs(B) from proc-struct(B)		
switch to k-stack(B)		
return-from-trap (into B)		
	restore regs(B) from k-stack(B)	
	move to user mode	
	jump to B's PC	
		Process B
		...

Figure 6.3: **Limited Direct Execution Protocol (Timer Interrupt)**

Рассмотрим на картинке процесс прерывания по времени. В этом примере процесс А запущен и работает, затем прерывается таймером. Оборудование сохраняет значения регистров (в стек ядра) и переключается в режим ядра. В обработчике прерывания ОС решает запустить следующим процесс Б. В это время будет вызвана функция switch(),

которая сохраняет текущие значения регистров процесса А (в структуру процесса А), восстанавливает регистры процесса Б (из структуры процесса Б) в стек ядра и затем переключает контексты. В частности это заключается в том что значение Stack Pointer меняется на указатель начала стека процесса Б. В конечном итоге ОС возвращается из прерывания, восстанавливает на процессоре значения регистров процесса Б и затем начинается его исполнение.

Обратите внимание, что в этом процессе 2 типа сохранения\восстановления регистров. Первый случается в момент срабатывания таймера - и в этом случае регистры пользовательского процесса сохраняются оборудованием, используя стек ядра этого процесса. Другой тип применяется когда ОС переключается между процессами. В этом случае значения регистров, хранящиеся в стеке ядра сохраняются программно (например ОС), но в этот раз сохраняется в структуру памяти процесса. И при завершении процедуры мы получаем состояние не как будто мы только что прервали процесс А, а будто-бы прервали процесс Б.

```
1   # void swtch(struct context **old, struct context *new);
2   #
3   # Save current register context in old
4   # and then load register context from new.
5   .globl swtch
6   swtch:
7     # Save old registers
8     movl 4(%esp), %eax  # put old ptr into eax
9     popl 0(%eax)        # save the old IP
10    movl %esp, 4(%eax)  # and stack
11    movl %ebx, 8(%eax)  # and other registers
12    movl %ecx, 12(%eax)
13    movl %edx, 16(%eax)
14    movl %esi, 20(%eax)
15    movl %edi, 24(%eax)
16    movl %ebp, 28(%eax)
17
18    # Load new registers
19    movl 4(%esp), %eax  # put new ptr into eax
20    movl 28(%eax), %ebp # restore other registers
21    movl 24(%eax), %edi
22    movl 20(%eax), %esi
23    movl 16(%eax), %edx
24    movl 12(%eax), %ecx
25    movl 8(%eax), %ebx
26    movl 4(%eax), %esp  # stack is switched here
27    pushl 0(%eax)       # return addr put in place
28    ret                 # finally return into new ctxt
```

Figure 6.4: **The xv6 Context Switch Code**

Ключевые моменты лекции:

1) CPU должен поддерживать как минимум 2 режима работы: ограниченный режим пользователя и привилегированный (неограниченный) режим ядра.
2) Типичные приложения работают в режиме пользователя и используют системные вызовы для перехода в режим ядра.
3) Инструкция trap сохраняет состояние регистров, изменяет статус оборудования на режим ядра и переходит в ОС в заранее определенное место - trap table.

4) Когда ОС завершает обслуживание системного вызова, она возвращается в программу пользователя через специальную инструкцию возвращения (return-from-trap), которая понижает привилегии и передает контроль следующей инструкции, которая стоит в программе сразу за вызовом trap.
5) Trap Table должна заполняться при загрузке ОС и не должна быть изменяема программами пользователя.
6) Запуская программу, ОС должна использовать аппаратные механизмы, чтобы убедиться, что программа пользователя не будет работать бесконечно. Сделать это можно через устройство называемое таймер прерываний. Такой подход к планированию использования CPU называется не кооперативным.
7) В некоторые моменты после срабатывания таймера прерываний или системного вызова, ОС может захотеть переключиться с одного запущенного процесса на другой. Это осуществляется через технику, которая называется переключение контекста (context switch).

Заметки

Прерывание (англ. interrupt) — сигнал от программного или аппаратного обеспечения, сообщающий процессору о наступлении какого-либо события, требующего немедленного внимания.
Trap - инструкция перехвата, которая осуществляет переключение из пользовательского режима в режим ядра
Есть градация в терминах по тому чем вызвано прерывание
trap - перехват с целью повышения привилегий

https://stackoverflow.com/questions/30419872/what-is-a-kernel-stack-used-for
https://stackoverflow.com/questions/12911841/kernel-stack-and-user-space-stack